Reflection Report

on

[Imitation of Television Remote Control System using STM32 MCU]

Student Credentials		
Name	PRN	Contact No.
Pritesh Pravin Purkar	202301060010	+91 9322770715
Nachiket Pravin Mahajan	202301060047	+91 8767145134
Shivraj Ramesh Nalavade	202301060008	+91 9028432689

Key Objectives:

1. Design and develop an embedded system that simulates a television remote control system using an STM32 microcontroller.
2. The system should allow users to change channels, adjust volume, and power ON/OFF using push-buttons.
3. The 7-segment display should indicate the selected channel number, while the LCD screen should provide detailed status updates such as “Channel 5,” “Volume: 10,” “TV ON”.
4. A relay will simulate TV power control, and a buzzer will generate alerts for power status changes and button presses.
5. Enable power ON/OFF, channel selection, volume adjustment, and mute control.

Intended Outcomes:

1. The system will function as an imitation of a television remote control unit, allowing users to switch between channels, adjust volume, mute, and turn the TV ON/OFF using push-buttons.
2. A 7-segment display will show the current channel number,
3. LCD screen will provide detailed information on channel selection, volume level, and mute status.
4. A relay will simulate the television power control, and a buzzer will confirm user inputs with sound alerts.
5. User can control a simulated TV system.

What real-world situation does this problem relate to?

Mimics how modern smart TV remotes work.

Demonstrates real-time embedded control used in consumer electronics.

Can be extended to IR remote simulation or home automation setups.

Simulates the working of remote control logic in smart TVs using buttons instead of IR signals.

Why is this problem important or worth solving?

Promotes hands-on learning of system integration, combining multiple components like LCD, buzzer, relay, and buttons to work together in a coordinated embedded system — a crucial skill for real-world product development.

What assumptions are made about the scenario?

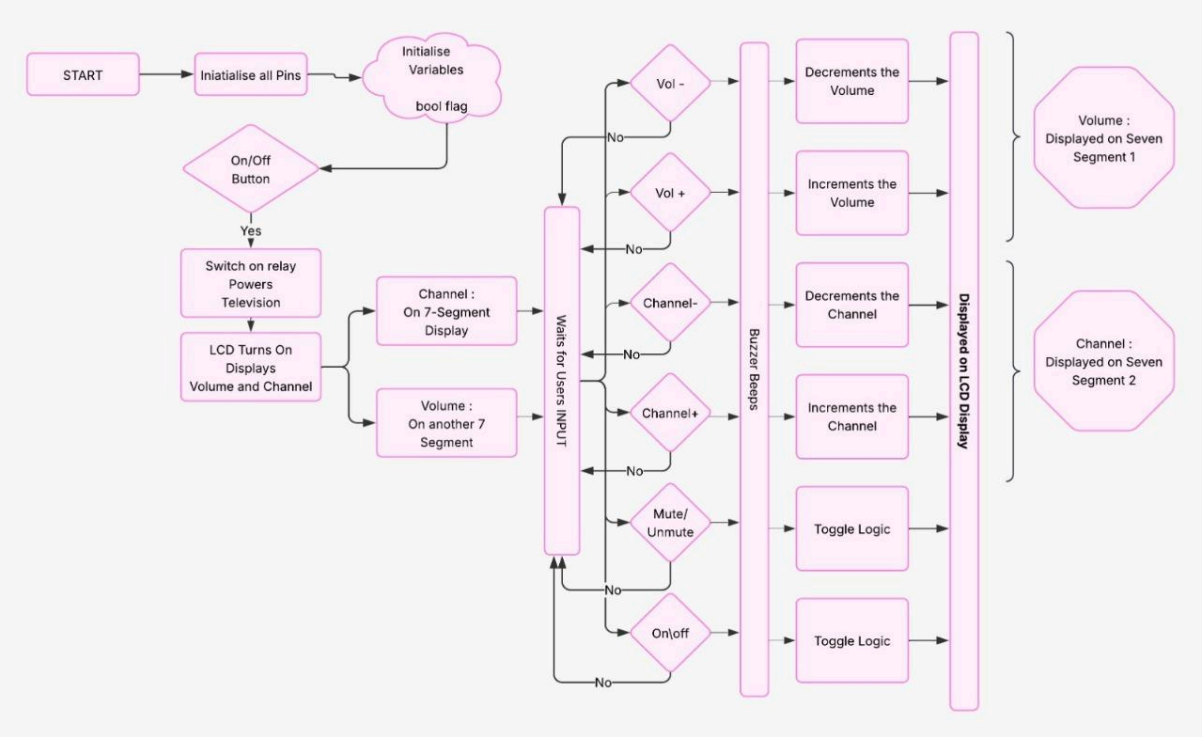
1. Only one device (TV) is being controlled.
2. System is user-operated directly (no wireless/IR).
3. Initial volume and channel have default starting values.
4. The TV remains in either fully ON or OFF state — no standby mode is considered.
5. All inputs are from physical push-buttons — no touch or remote interface is used.
6. Power supply is stable and sufficient for operating all components without fluctuation.

List of Components Used

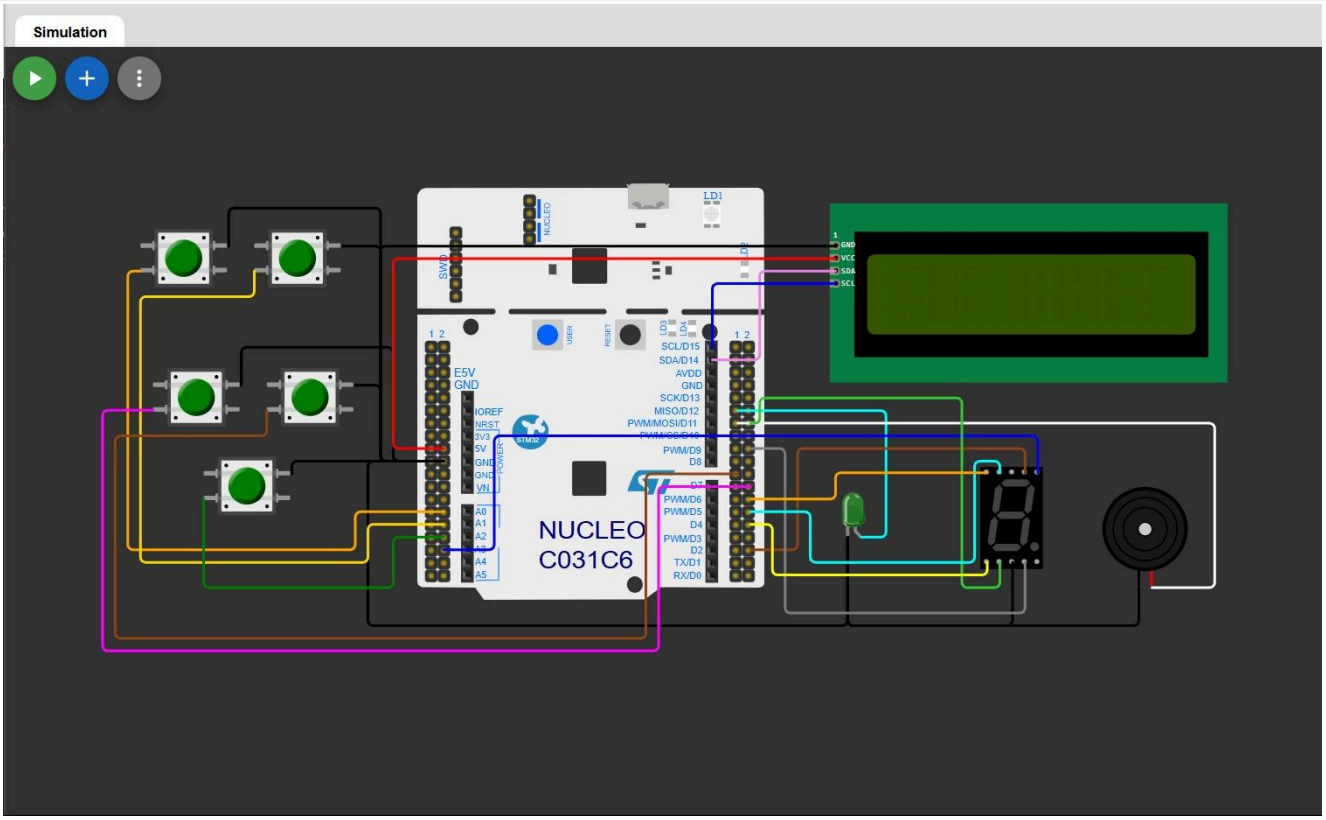
Name of Component	Specs	Quantity	Input/ Output/ Processing Element
STM32 Microcontroller	**Core**: ARM Cortex-M4 with FPU (Floating Point Unit) **Operating Frequency**: Up to 100 MHz **Flash Memory**: 512 KB **SRAM**: 128 KB **CPU Performance**: 125 DMIPS **Supply Voltage (VDD)**: 1.7 V to 3.6 V **Operating Temperature**: -40°C to +85°C **Communication Interfaces**: 3x SPI, 3x I2C, 4x USART/UART,USB 2.0 Full Speed,SDIO,CAN	1	Processing Element
Push-Buttons	**Type**: SPST, normally open (closes only when pressed). **Voltage & Current Rating**: Up to 12V DC, 50 mA **Debounce Requirement**: Mechanical bounce ~5–10 ms	5	Input
7-Segment Display	**Type**: Common Cathode configuration. **Forward Voltage per Segment**: ~1.8V to 2.2V **Current per Segment**: Typically 5–20 mA for proper brightness	1	Output
Buzzer	**Operating Voltage**: Typically 3.3V to 5V **Sound Frequency**: Fixed, usually around 2 kHz to 4 kHz	1	Output
Relay	**Control Voltage**: 3.3V or 5V **Load Capacity**: Up to 10A at 250V AC or 10A at 30V DC. **Trigger Type**: Active LOW or HIGH	1	Processing Flement
i2C LCD Display	**Interface**: I2C (uses only 2 data pins: SDA & SCL). **Operating Voltage**: 5V.. **Address**: **0x27**	1	Output

IR Receiver	Operating Voltage: 2.7V to 5.5V **Carrier Frequency**: 38 kHz **Reception Distance**: Up to 10–15 meters **Viewing Angle**: ~ ±45 degrees for reliable signal reception **Output**: Active LOW digital signal when IR pulses are received.	1	Input

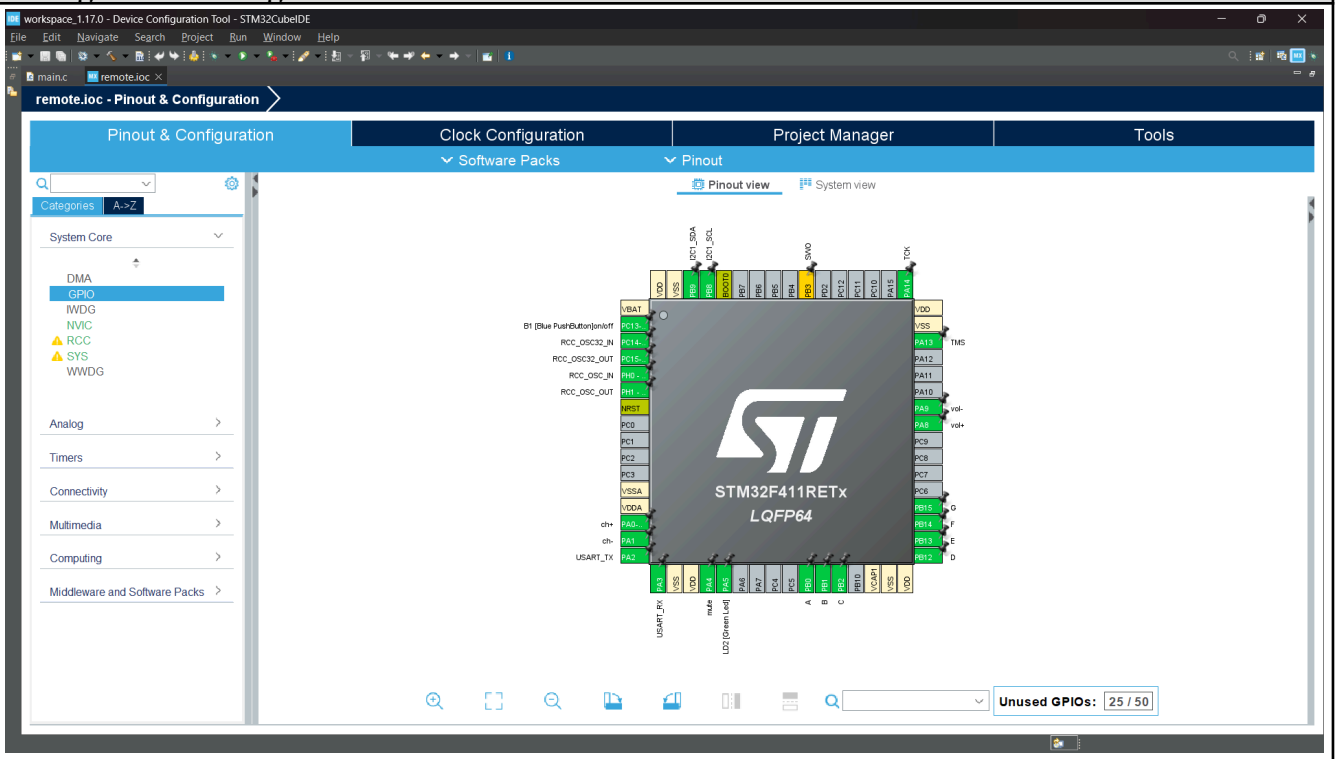
System Architecture – Block Diagram



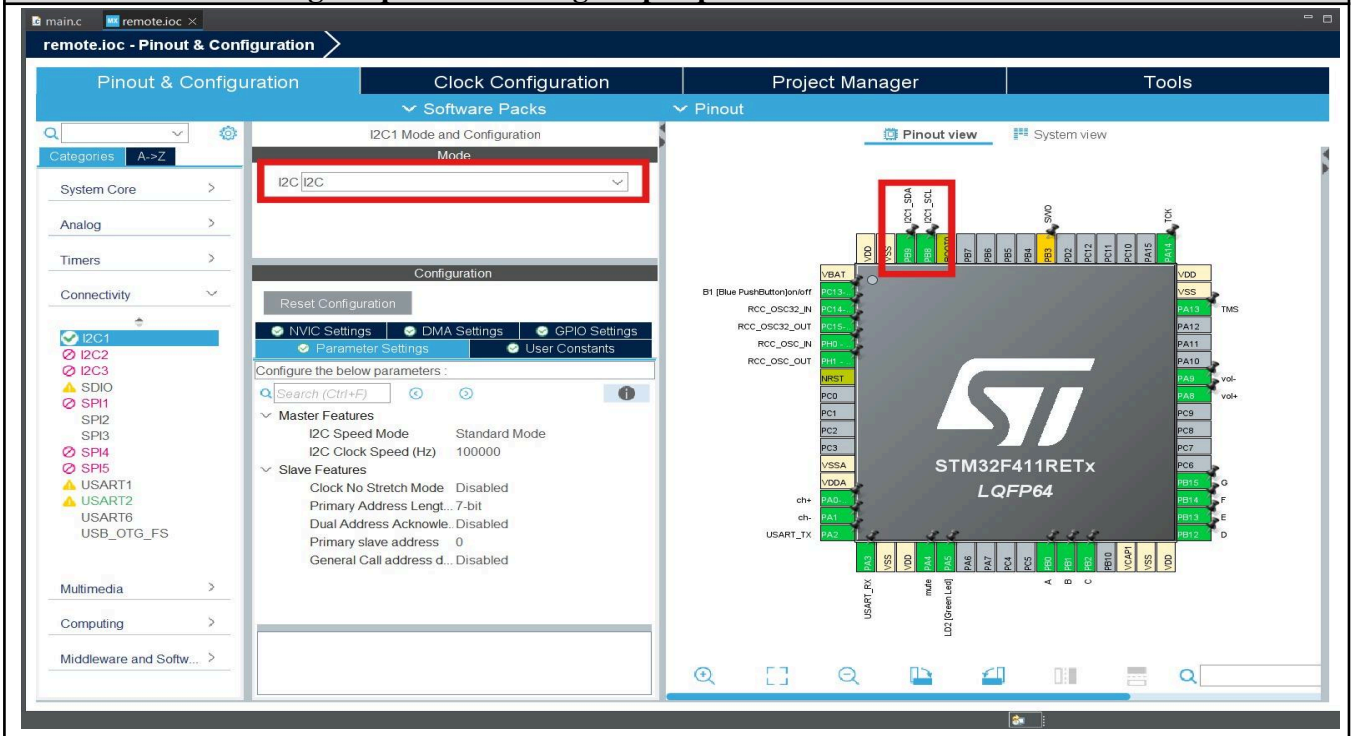
Circuit Diagram of the Implemented System



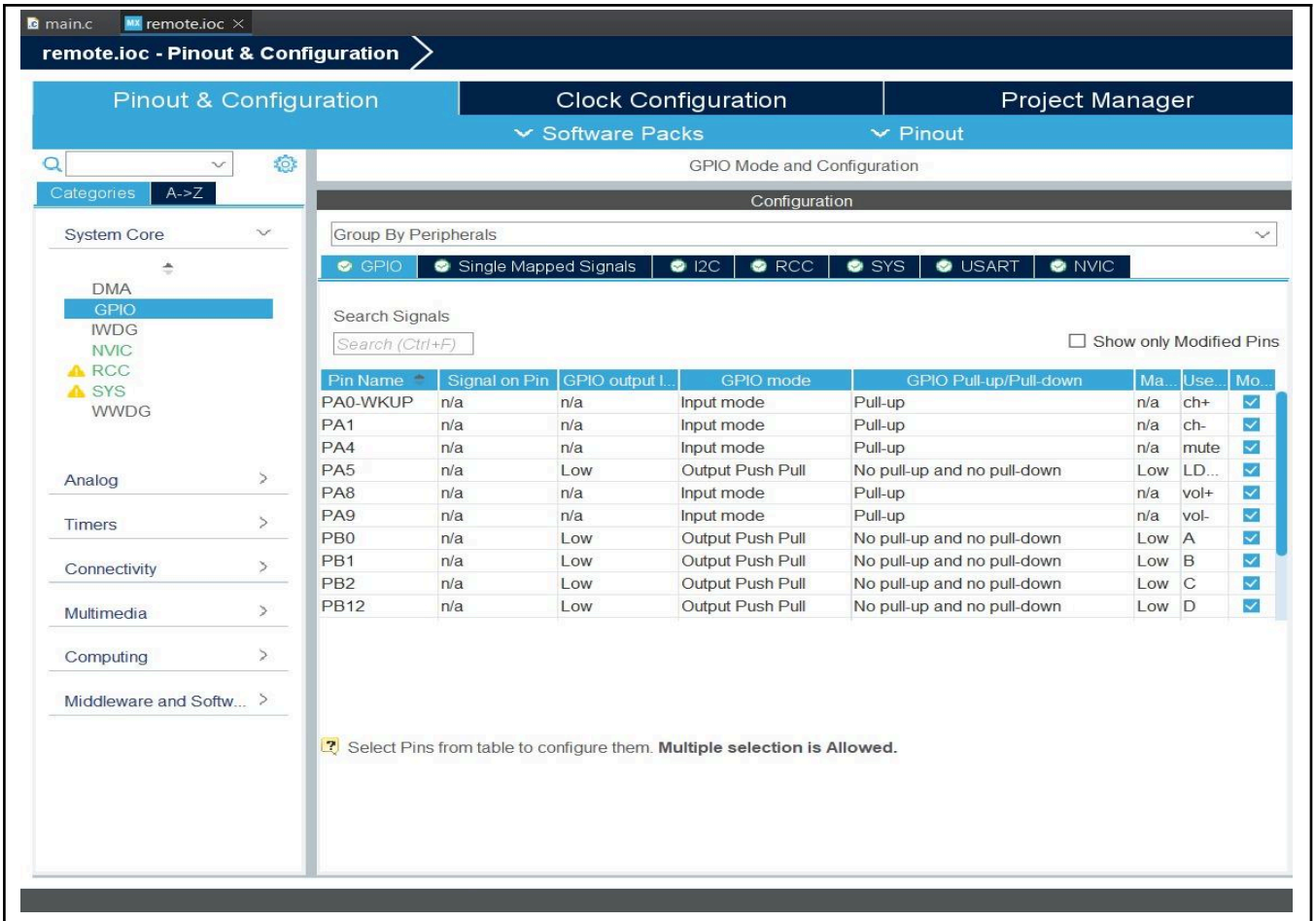
Configured Pin Diagram from STM32 CubeIDE



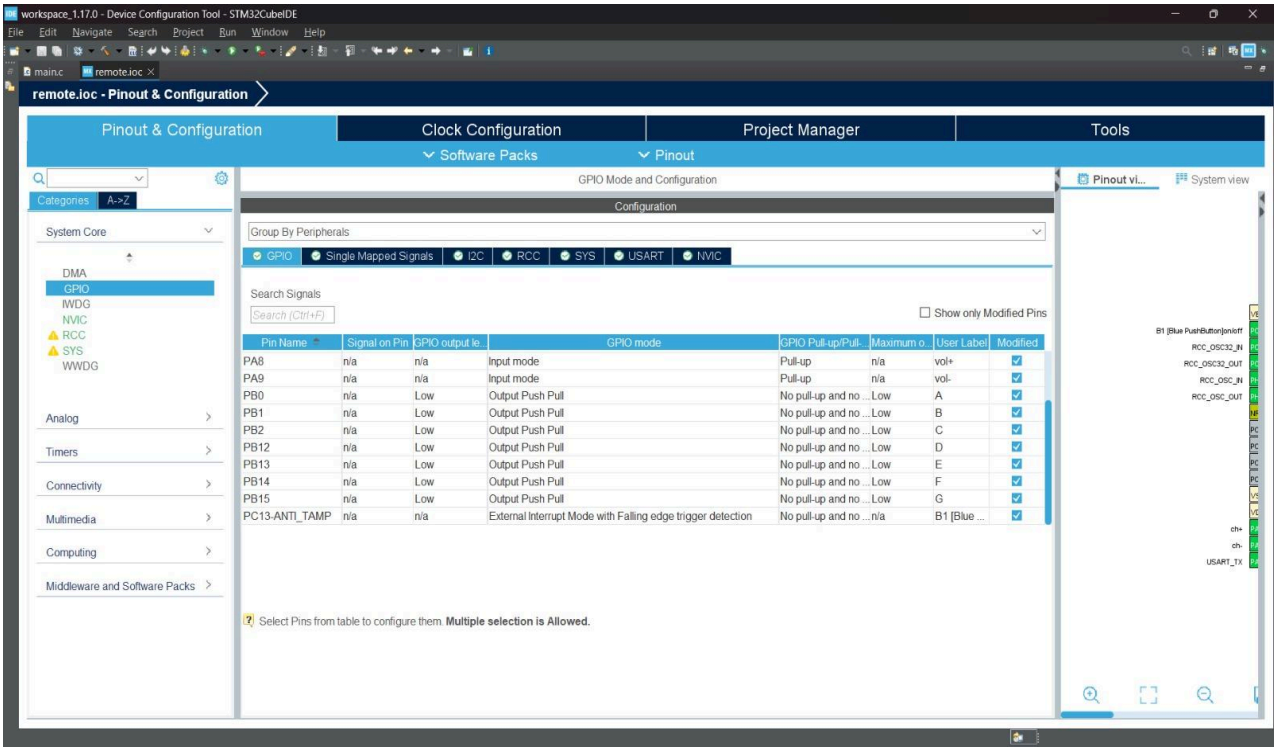
Mention all the settings required to configure peripherals in STM32 CubeIDE



In the connectivity category, we have selected I2C for communication with the LCD display. By default, the SDA pin is PB6 and the SCL pin is PB7, but we have changed them to I2C1_SDA on pin PB9 and I2C1_SCL on pin PB8

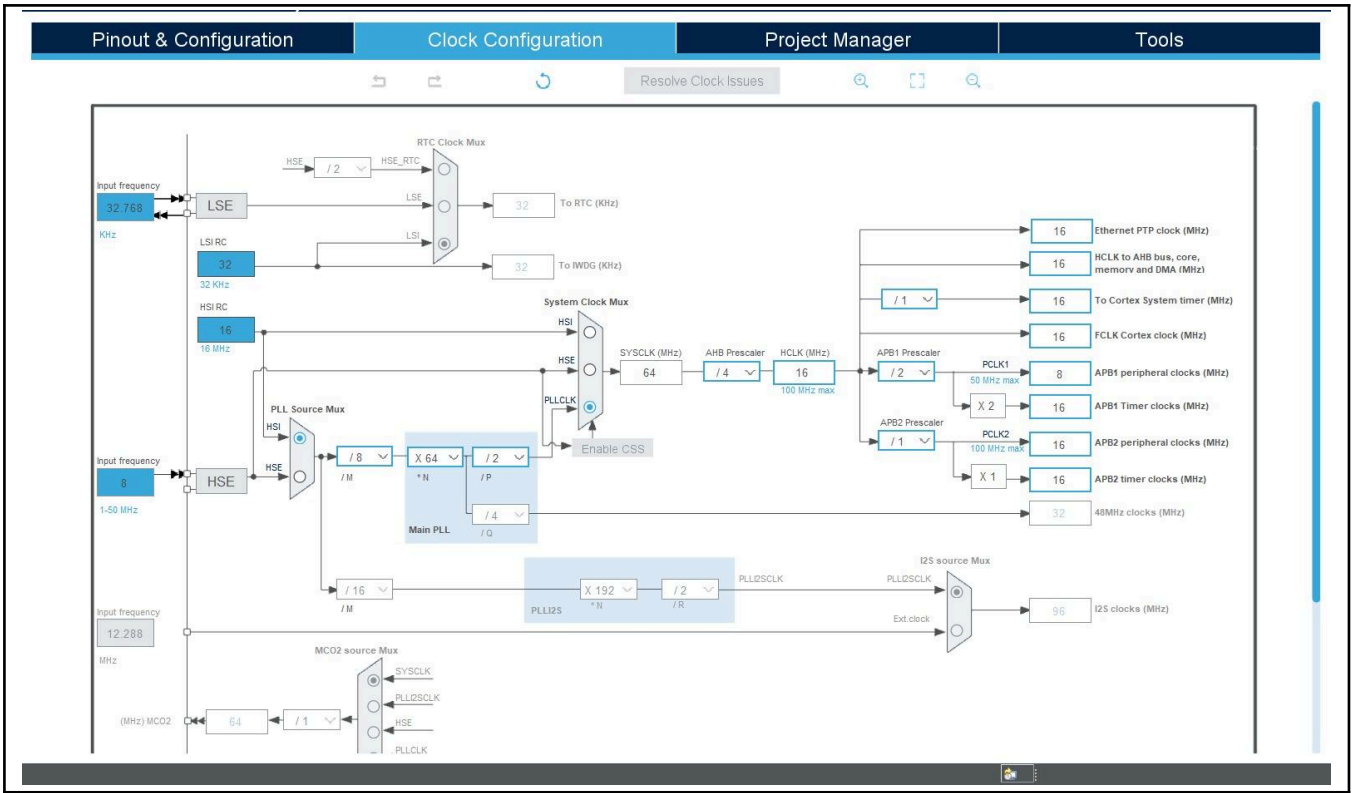


.



System Core GPIO pin configuration in our setup :

- **PA0** is connected to the **CH+ button**, set as input mode with pull-up.
- **PA1** is connected to the **CH− button**, also configured as input mode with pull-up.
- **PA4** is connected to the **MUTE button**, configured in input mode with pull-up.
- **PA5** is configured as a GPIO output (default low) using output push-pull mode with no pull-up or pull-down.
- **PA8** is connected to an LED, set as input mode with pull-up.
- **PA9** is used for Volume+ and Volume−, configured as input mode with pull-up.
- **PC13** is configured as an external interrupt input with falling edge trigger detection.
- This serves as **Button 1** on the STM32 board for **ON/OFF** functionality.
- Pins **PB1**, **PB2**, **PB12**, **PB13**, **PB14**, and **PB15** are connected to a 7-segment display, all configured as:
    - GPIO output (default low)
    - Output push-pull mode
    - No pull-up or pull-down
    - Representing segments a, b, c, d, e, f, g



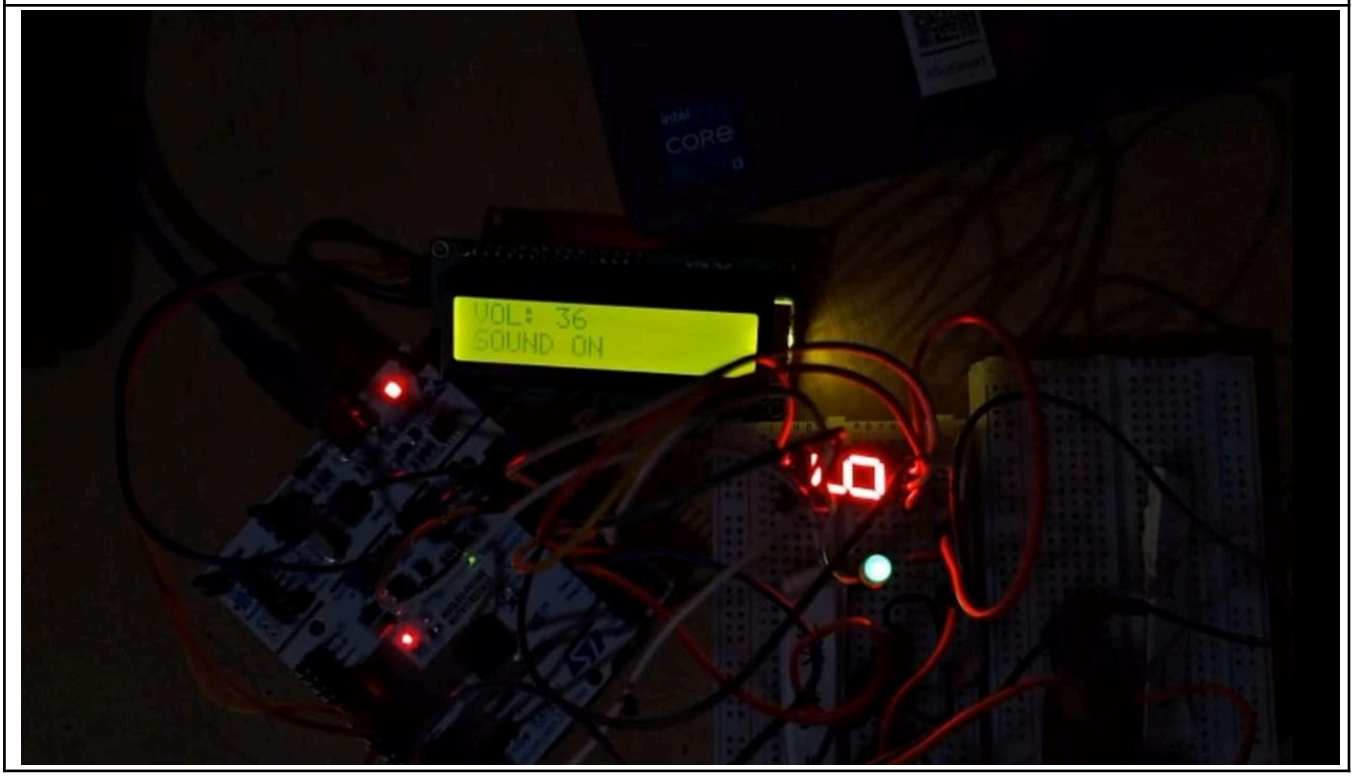
In the **Clock Configuration**, we have changed the **HCLK** to **16 MHz** to support the **LCD display**.

Challenges faced

1. Lack of proper components in College.
2. Faced a lot of problems in interfacing LCD normally, so I attached an i2C module to it and then worked.
3. STM 32 Cube IDE is very compact and the UI/UX of it quite complex, so faced a little issue.

Photo Gallery


MIT ACADEMY OF ENGINEERING, ALANDI - PUNE


VOL: 36
SOUND ON



Appendices:

Firmware – Source STM32 CubeIDE

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
```

```
#include "i2c.h"
#include "usart.h"
#include "gpio.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "i2c_lcd.h"
#include "stdio.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
#define SEG_GPIO_PORT GPIOB
#define SEG_A_PIN GPIO_PIN_0
#define SEG_B_PIN GPIO_PIN_1
#define SEG_C_PIN GPIO_PIN_2
#define SEG_D_PIN GPIO_PIN_12
#define SEG_E_PIN GPIO_PIN_13
#define SEG_F_PIN GPIO_PIN_14
#define SEG_G_PIN GPIO_PIN_15

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/

/* USER CODE BEGIN PV */
uint8_t isOn = 0;
uint8_t volume = 30;
uint8_t channel = 1;
uint8_t isMuted = 0;
char displayBuffer[16];

const uint8_t segment_digits[10][7] = {
    {1,1,1,1,1,1,0}, // 0
    {0,1,1,0,0,0,0}, // 1
    {1,1,0,1,1,0,1}, // 2
    {1,1,1,1,0,0,1}, // 3
    {0,1,1,0,0,1,1}, // 4
    {1,0,1,1,0,1,1}, // 5
    {1,0,1,1,1,1,1}, // 6
```

```
    {1,1,1,0,0,0,0}, // 7
    {1,1,1,1,1,1,1}, // 8
    {1,1,1,1,0,1,1}  // 9
};
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void display_digit(uint8_t digit) {
    if (digit > 9) return;  // Only handle digits 0-9

    HAL_GPIO_WritePin(SEG_GPIO_PORT, SEG_A_PIN, segment_digits[digit][0] ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(SEG_GPIO_PORT, SEG_B_PIN, segment_digits[digit][1] ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(SEG_GPIO_PORT, SEG_C_PIN, segment_digits[digit][2] ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(SEG_GPIO_PORT, SEG_D_PIN, segment_digits[digit][3] ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(SEG_GPIO_PORT, SEG_E_PIN, segment_digits[digit][4] ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(SEG_GPIO_PORT, SEG_F_PIN, segment_digits[digit][5] ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(SEG_GPIO_PORT, SEG_G_PIN, segment_digits[digit][6] ?
GPIO_PIN_SET : GPIO_PIN_RESET);
}
/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();
```

```
  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  MX_I2C1_Init();
  /* USER CODE BEGIN 2 */
  lcd_init();
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
//          main
          // Check ON/OFF button (PC13)
             if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
                HAL_Delay(200); // debounce
                isOn = !isOn;
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, isOn ? GPIO_PIN_SET :
GPIO_PIN_RESET);
                lcd_clear();
                HAL_Delay(500);
                display_digit(0);
             }

             if (isOn) {
                // Channel UP (PA0)

                if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET) {
                   if (channel < 9) channel++;
                   display_digit(channel);
                   char msg[10];
                   int len = sprintf(msg, "CH: %d\n", channel); // convert channel to string
                   HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, HAL_MAX_DELAY);
                   HAL_Delay(300); // debounce
                }
```

```
            // Channel DOWN (PA1)
            if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_RESET) {
                if (channel > 1) channel--;
                display_digit(channel);
                char msg[10];
                int len = sprintf(msg, "CH: %d\n", channel); // convert channel to string
                HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, HAL_MAX_DELAY);
                HAL_Delay(300);
            }

            // Volume UP (PA8)
            if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_RESET) {
                if (volume < 100 && !isMuted) volume++;
                HAL_Delay(300);
            }

            // Volume DOWN (PA9)
            if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9) == GPIO_PIN_RESET) {
                if (volume > 0 && !isMuted) volume--;
                HAL_Delay(300);
            }

            // Mute toggle (PA4)
            if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_4) == GPIO_PIN_RESET) {
                isMuted = !isMuted;
                HAL_Delay(300);
            }

            // LCD Display
            lcd_put_cursor(0, 0);
            sprintf(displayBuffer, "VOL: %d     ", isMuted ? 0 : volume);
            lcd_send_string(displayBuffer);

            lcd_put_cursor(1, 0);
            lcd_send_string(isMuted ? "MUTED       " : "SOUND ON    ");
        } else {
            lcd_put_cursor(0, 0);
            lcd_send_string("System OFF     ");
            lcd_put_cursor(1, 0);
            lcd_send_string("Press ON Btn");
        }
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
```

```
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 8;
  RCC_OscInitStruct.PLL.PLLN = 64;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 4;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV4;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
```

```
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

Proteus Circuit *(if applicable)*

